

VBA Programming Basics – Practical Guide

Beginner-friendly practicals covering Excel setup, VBA Editor, creating a macro, MsgBox, Range, Cells, formulas, formatting, variables, IF conditions, and loops.

Learning Objectives

- Open and use the VBA Editor.
- Create a VBA Module and Sub procedure.
- Run a VBA macro using F5 or the Run button.
- Write data into Excel using Range and Cells.
- Use VBA to calculate a total.
- Format Excel cells using VBA.
- Use variables and data types.
- Use IF...Else conditions.
- Use a For loop to process multiple rows.

Before You Start

Save your workbook as an Excel Macro-Enabled Workbook (*.xlsm). This allows VBA code to be saved with the workbook.

File → Save As → Excel Macro-Enabled Workbook (*.xlsm)

Practical 1: Create a Simple VBA Macro

Step 1 – Open Excel and enter the following data:

A	B
Name	Marks
Rahul	75
Priya	82
Amit	68
Neha	91

Step 2 – Enable the Developer Tab

- Click File.
- Click Options.
- Select Customize Ribbon.
- On the right side, tick Developer.
- Click OK.
- You should now see Developer on the Excel ribbon.

Step 3 – Open the VBA Editor

Click Developer → Visual Basic, or press Alt + F11.

Step 4 – Insert a Module

- In the VBA Editor, click Insert.
- Select Module.
- A blank code window will appear.

Step 5 – Write Your First VBA Code

```
Sub MyFirstMacro()  
  
    MsgBox "Hello! This is my first VBA Macro."  
  
End Sub
```

What does the code mean?

Sub MyFirstMacro() – Starts a macro named MyFirstMacro.

MsgBox – Displays a message box to the user.

End Sub – Ends the macro.

Step 6 – Run the Macro

- Place the cursor anywhere inside the macro.
- Press F5, or click the green Run button.
- A message box will appear: Hello! This is my first VBA Macro.

Practical 2: Write Data Using VBA

Add this macro in the same Module or a new Module:

```
Sub EnterData()  
  
    Range("D1").Value = "Employee"  
    Range("E1").Value = "Salary"  
  
    Range("D2").Value = "Rahul"  
    Range("E2").Value = 30000  
  
    Range("D3").Value = "Priya"  
    Range("E3").Value = 35000  
  
    Range("D4").Value = "Amit"  
    Range("E4").Value = 28000  
  
End Sub
```

Expected output:

Employee	Salary
Rahul	30000
Priya	35000
Amit	28000

Understanding Range

Range("D2").Value = "Rahul" means VBA puts the value Rahul into cell D2.

Practical 3: Calculate Total

```
Sub CalculateTotal()  
  
    Range("F1").Value = "Total Salary"  
  
    Range("F2").Formula = "=SUM(E2:E4)"  
  
End Sub
```

After running the macro, F2 contains the total salary: 93000.

Practical 4: Format Cells Using VBA

```
Sub FormatData()  
  
    Range("D1:F1").Font.Bold = True  
    Range("D1:F1").Interior.Color = vbYellow  
  
    Range("D1:F4").Borders.LineStyle = xlContinuous  
  
    Columns("D:F").AutoFit  
  
End Sub
```

- Font.Bold = True makes the header bold.
- Interior.Color = vbYellow adds a yellow background.
- Borders.LineStyle adds borders.
- Columns("D:F").AutoFit adjusts the column widths automatically.

Practical 5: Variables and Data Types

```
Sub VariablesExample()  
  
    Dim employeeName As String  
    Dim salary As Double  
    Dim age As Integer  
    Dim isActive As Boolean  
  
    employeeName = "Rahul"  
    salary = 35000  
    age = 25  
    isActive = True  
  
    MsgBox "Name: " & employeeName & vbNewLine & _  
        "Salary: " & salary & vbNewLine & _  
        "Age: " & age & vbNewLine & _  
        "Active: " & isActive  
  
End Sub
```

Data Type	Purpose	Example
String	Text	"Rahul"
Integer	Whole number	25
Long	Large whole number	100000
Double	Decimal number	45000.75
Boolean	True/False	True
Date	Date/time	30-09-2026

Practical 6: IF Condition + Cells

```
Sub CheckSales()  
  
    If Range("C2").Value >= 5000 Then  
        Range("D2").Value = "Good"  
    Else  
        Range("D2").Value = "Low"  
    End If  
  
End Sub
```

If C2 is 5000 or more, D2 becomes Good. Otherwise, D2 becomes Low.

Practical 7: For Loop – Pass/Fail

Suppose marks are in column B and you want Pass/Fail in column C. The loop checks rows 2 through 5 automatically.

```
Sub CheckResult()  
  
    Dim i As Integer  
  
    For i = 2 To 5  
  
        If Cells(i, 2).Value >= 50 Then  
            Cells(i, 3).Value = "Pass"  
        Else  
            Cells(i, 3).Value = "Fail"  
        End If  
  
    Next i  
  
End Sub
```

Name	Marks	Result
Rahul	75	Pass
Priya	82	Pass
Amit	68	Pass
Neha	91	Pass

How the For Loop Works

```
For i = 2 To 5  
    ...  
Next i
```

The variable i takes the values 2, 3, 4, and 5. Cells(i, 2) means column B, while Cells(i, 3) means column C.

Quick Reference: Range vs Cells

Method	Example	Meaning
Range	Range("A1")	Cell A1
Range	Range("A1:B5")	A1 through B5
Cells	Cells(2,1)	Row 2, Column 1 = A2
Cells	Cells(2,3)	Row 2, Column 3 = C2

Practice Task

Create a worksheet with Student Name and Marks. Write a VBA macro that loops through all students, checks whether marks are at least 50, and writes Pass or Fail in a Result column. Then format the header using bold text, background color, borders, and AutoFit.

Important VBA Tips

- Save the workbook as .xlsm.
- Use Alt + F11 to open the VBA Editor.
- Use Insert → Module to create a standard module.
- Use F5 to run a macro from the VBA Editor.
- Use Range when you know the cell address.
- Use Cells when working with row and column numbers, especially in loops.
- You can keep multiple Sub procedures in one Module.